

MALLA REDDY TECHNICAL CAMPUS

A constituent Unit of MALLA REDDY VISHWAVIDYAPEETH

Deemed to be University

Approved by UGC & AICTE New Delhi

Maisammaguda, Medchal (Dist), Hyderabad – 500100, Telangana



SMART INVENTORY MANAGEMENT

NAME: RUPASRI RACHARLA, ROOPIKA
GANGIREDDY, SUGUNA RUDARAM, S.
LOITH

TEAM NAME: Coders

TEAM NO: 17

Problem Statement

- To select products to stock under limited storage capacity while maximizing the expected benefit.

Inputs :

- Number of products
- Product name
- Storage space required by each product
- Expected benefit/ value of each product
- Total available storage capacity.

Outputs:

- Selected products to stock
- Total storage used
- Maximum expected benefit

Constraints:

- Each product can be selected **only once** (0 or 1).
- Total storage used **cannot exceed the available capacity**.
- Storage requirements must be **non-negative**.
- Expected benefits must be **non-negative**.
- The solution should **maximize total expected benefit** within the storage limit.

Proposed Approach

Algorithm Paradigm: Dynamic Programming (DP)

Approach: Use the 0/1 Knapsack algorithm to select products that maximize expected benefit without exceeding the available storage capacity.

Why DP suits the problem: The problem has overlapping subproblems and optimal substructure. DP avoids repeatedly solving the same smaller problems and efficiently finds the optimal product combination.

Alternatives considered:

- **Brute Force:** Checks every possible combination, but has $O(2^n)$ time complexity.
- **Greedy:** Faster, but does not always guarantee the optimal solution for 0/1 Knapsack.

Proposed solution: Dynamic Programming with $O(nW)$ time and $O(nW)$ space complexity.

Objectives

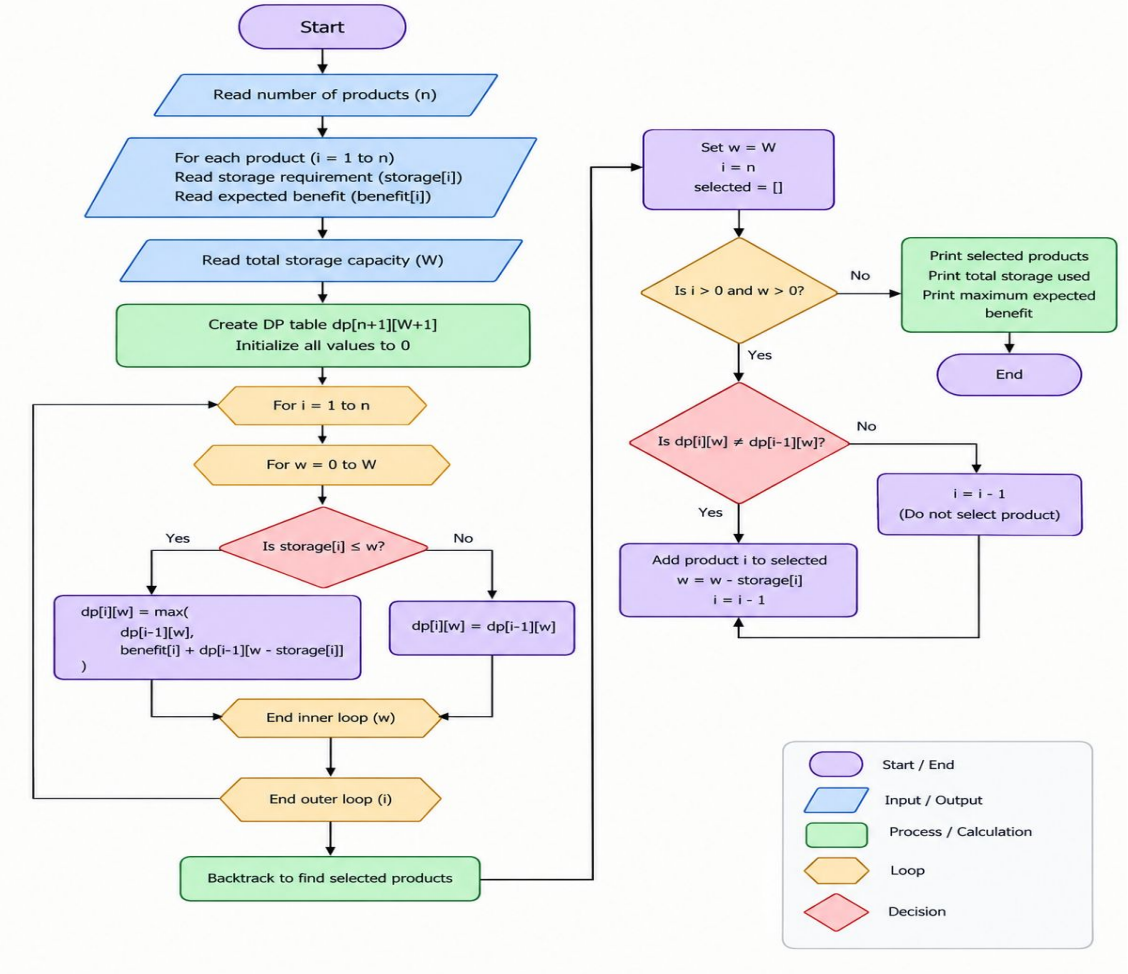
Optimize inventory selection by identifying which products should be stocked within limited storage capacity.

Maximize business value by prioritizing products based on their expected demand, importance, or benefit.

Reduce storage wastage and inefficient stocking by making data-driven inventory decisions.

Provide a smart and automated decision-making system that helps businesses manage limited warehouse space efficiently.

Pseudocode



Complexity

Complexity	Best Case	Average Case	Worst Case
Time	$O(nW)$	$O(nW)$	$O(nW)$
Space	$O(nW)$	$O(nW)$	$O(nW)$

Time: $O(nW)$

The DP table has $(n+1) \times (W+1)$ cells, and the algorithm processes each cell once using the nested loops. Therefore, the time complexity is $O(nW)$ in all cases.

Space: $O(nW)$

The program stores the entire DP table containing $(n+1) \times (W+1)$ entries, so space complexity is $O(nW)$.

The final backtracking to find the selected products takes $O(n)$, but this is smaller than the DP computation, so the overall complexity remains $O(nW)$.

Implementation

To establish this project we used few resources such as:

1. **Python** - to write and implement code using the 0/1 Knapsack Problem
2. **Visual Studio Code**- The actual Python code was written here as it hosts the code to be able to run as well.
3. **VS Code Integrated terminal** - Used to provide inputs and test the program.
4. **GitHub**- Repository used to host the code so that multiple people can access it.
5. **Dynamic Programming** - Algorithm design technique used to solve the optimization problem.

Results and Test cases

Approach	Time Complexity	Description
Naive / Brute Force	$O(2^n)$	Checks every possible combination of products
Dynamic Programming	$O(nW)$	Stores results of smaller subproblems in a DP table

```
Enter storage space: 1
Enter expected benefit: 0
Enter product name: pencil
Enter storage space: 3
Enter expected benefit: 2
Enter product name: books
Enter storage space: 4
Enter expected benefit: 1
Enter product name: pouch
Enter storage space: 2
Enter expected benefit: 1
Enter storage capacity: 5
```

```
Selected Products:
candle - Space: 2 Benefit: 1
pencil - Space: 3 Benefit: 2
```

```
Total Storage Used: 5
Maximum Expected Benefit: 3
```

Test Case 1

Input: 4 products, capacity = 5

Output: Candle + Pencil

Storage Used: 5

Maximum Benefit: 3

Test Case 2

Input: 2 products, capacity = 1

Output: Bottle

Storage Used: 1

Maximum Benefit: 2

```
Enter number of products: 2
Enter product name: bottle
Enter storage space: 1
Enter expected benefit: 2
Enter product name: phone
Enter storage space: 4
Enter expected benefit: 2
Enter storage capacity: 1
```

```
Selected Products:
bottle - Space: 1 Benefit: 2
```

```
Total Storage Used: 1
Maximum Expected Benefit: 2
```

Conclusion and Future Scope

Outcomes

- Successfully identifies the **optimal combination of products** within limited storage capacity.
- **Maximizes expected benefit** while ensuring the storage limit is not exceeded.
- Demonstrates how **0/1 Knapsack with Dynamic Programming** can be applied to real-world inventory management.
- Provides an interactive system where users can **enter product details and receive the optimal inventory selection**.

Possible Improvements

- Add a **database** to store and manage product information.
- Include real-time factors such as **product demand, sales trends, and stock levels**.
- Add a **user-friendly web/mobile interface** instead of terminal-based input.
- Support **multiple warehouses or storage locations** with different capacities.
- Add **data visualization and reports** to help users understand inventory decisions.
- Incorporate **real-world constraints**, such as product expiration dates, minimum stock requirements, and changing product demand.

References

- [1] E. A. Goncalves de Souza, T. Mlinar, M. Van den Broeke, and S. Creemers, “Dynamic programming in inventory management: A review,” *Computers & Operations Research*, vol. 183, Art. no. 107164, 2025, doi: 10.1016/j.cor.2025.107164.
- [2] M. Demaine, J. Ku, and J. Solomon, “Introduction to Algorithms: 6.006,” Massachusetts Institute of Technology, *MIT OpenCourseWare*, 2020.
- [3] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.
- [4] Python Software Foundation, “The Python Tutorial,” *Python Documentation*. [Online]. Available: [Python Documentation](#).